

Stanford CS336: Language Modeling from Scratch

Lecture 10: Inference

Spring 2026 中文精讲笔记

主讲: Percy Liang



频道: Stanford Online

发布日期: 2026-05-11

时长: 1:25:30

视频: <https://www.youtube.com/watch?v=EfM546A79aM>

官方主页: <https://cs336.stanford.edu/>

目录

1 导读：问题、证据与阅读方法	2
2 推理负载和服务指标	2
3 Transformer 资源核算	6
4 KV cache 与 prefill/decode	10
5 延迟吞吐和显存模型	14
6 GQA 与 MLA	18
7 量化剪枝和蒸馏	22
8 推测解码	26
9 连续批处理与分页注意力	30
10 代码化复现：最小机制	34
11 总结与延伸	35
12 资料来源与审计边界	35

1 导读：问题、证据与阅读方法

本讲主问题

为什么自回归 decode 通常受内存带宽限制，架构和服务系统又怎样共同降低每个输出 token 的成本？

推理优化同时处理模型内的算子、KV cache 的状态和在线请求的调度；单点微优化不能替代端到端核算。本笔记以课程视频、人工字幕和 Stanford 官方讲义或代码为共同证据源。图像均来自每十五秒抽帧后的人工接触表审查，时间脚注可回到原视频复核；公式区分课程机制关系和本笔记的决策分析框架。学习时建议先读主问题，再沿“机制-资源-失败模式-验证指标”四列整理自己的实验。

首先排除的误读

只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

1.1 关键数字核对

主张	数值或关系	视频时间
OpenAI 每日处理 token	约 8.6T/day	00:03:00
DeepSeek v4 训练 token	32T	00:03:00
H100 bf16 峰值	989 TFLOP/s	00:17:00
H100 HBM 带宽	3.35 TB/s	00:17:00
H100 机器平衡算术强度	约 295 FLOP/byte	00:18:00
批一 decode 算术强度	约 1	00:20:00
Llama 13B 示例层数	40	00:40:00

1.2 本章小结

本章建立证据边界：视频解释提供因果和强调，官方材料提供公式与代码，精选帧提供可核查的视觉状态。后文每个主题都同时回答“为何重要、怎样工作、怎样失败、怎样验证”，避免把课程压缩成术语目录。

2 推理负载和服务指标

本节路线

本节依次处理：推理用于产品评测和强化学习采样、首 token 时间单请求延迟与吞吐指标、动态请求的到达时间前缀和长度差异。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I = \frac{\text{FLOPs}}{\text{bytes}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

$$G_1 = \frac{B_1}{C_1 + \lambda R_1 + \varepsilon}$$

- B_1 是本节方法带来的质量、吞吐或覆盖收益。
- C_1 是新增计算、显存、通信、数据或标注成本。
- R_1 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_1 \leq C_{\max}, \quad R_1 \leq R_{\max}, \quad Q_1 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

2.1 推理用于产品评测和强化学习采样

核心判断。“推理用于产品评测和强化学习采样”是“推理负载和服务指标”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 1 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“推理用于产品评测和强化学习采样”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

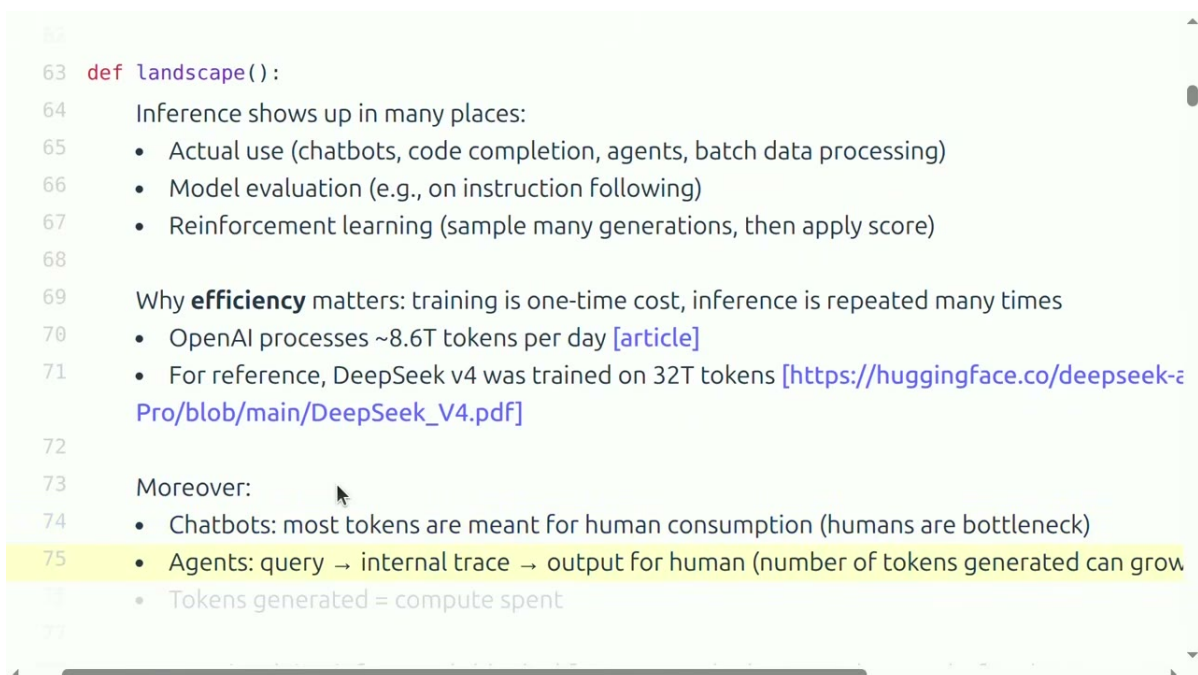


图 1: 推理用于产品评测和强化学习采样¹

2.2 首 token 时间单请求延迟与吞吐指标

核心判断。“首 token 时间单请求延迟与吞吐指标”是“推理负载和服务指标”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 2 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“首 token 时间单请求延迟与吞吐指标”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹视频画面时间区间：00:03:00–00:03:15。

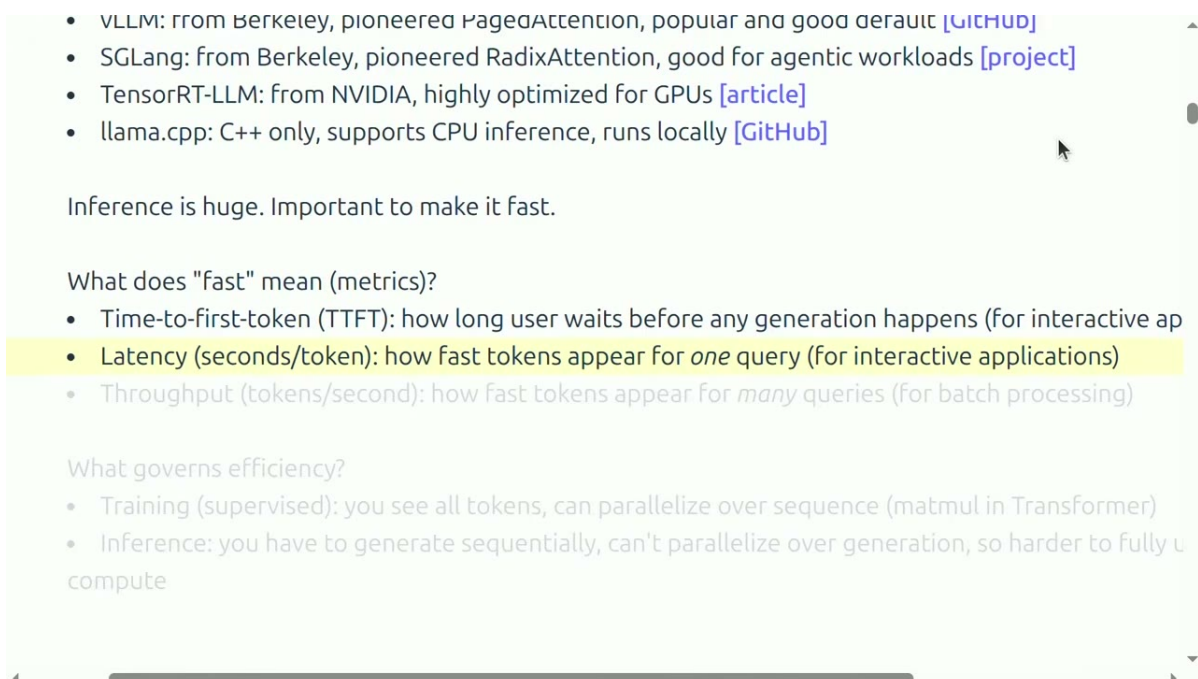


图 2: 首 token 时间单请求延迟与吞吐指标²

2.3 动态请求的到达时间前缀和长度差异

核心判断。“动态请求的到达时间前缀和长度差异”是“推理负载和服务指标”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 3 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“动态请求的到达时间前缀和长度差异”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²视频画面时间区间：00:06:00–00:06:15。

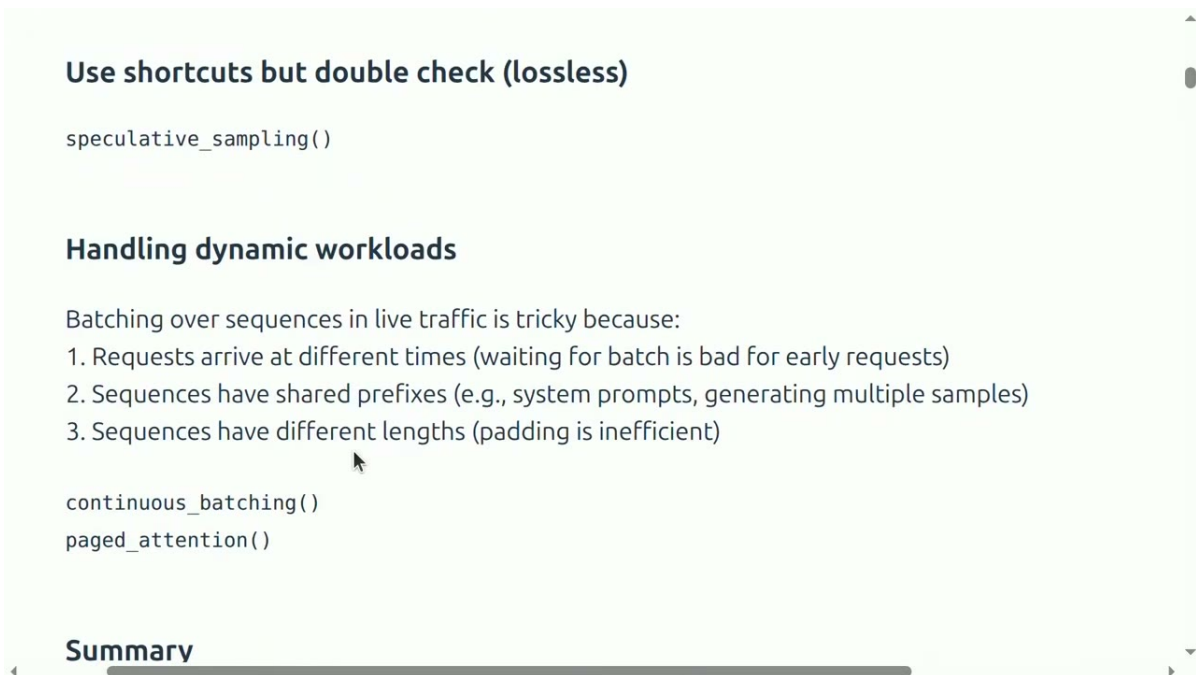


图 3: 动态请求的到达时间前缀和长度差异³

2.4 本章小结

“推理负载和服务指标”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

3 Transformer 资源核算

本节路线

本节依次处理：Transformer 推理张量维度记号、矩阵乘 FLOPs 与高带宽内存读写核算、H100 算术强度阈值约二百九十五。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I_{\text{MLP}} \approx B$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

³视频画面时间区间：00:08:45–00:09:00。

$$G_2 = \frac{B_2}{C_2 + \lambda R_2 + \epsilon}$$

- B_2 是本节方法带来的质量、吞吐或覆盖收益。
- C_2 是新增计算、显存、通信、数据或标注成本。
- R_2 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_2 \leq C_{\max}, \quad R_2 \leq R_{\max}, \quad Q_2 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

3.1 Transformer 推理张量维度记号

核心判断。“Transformer 推理张量维度记号”是“Transformer 资源核算”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 4 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“Transformer 推理张量维度记号”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

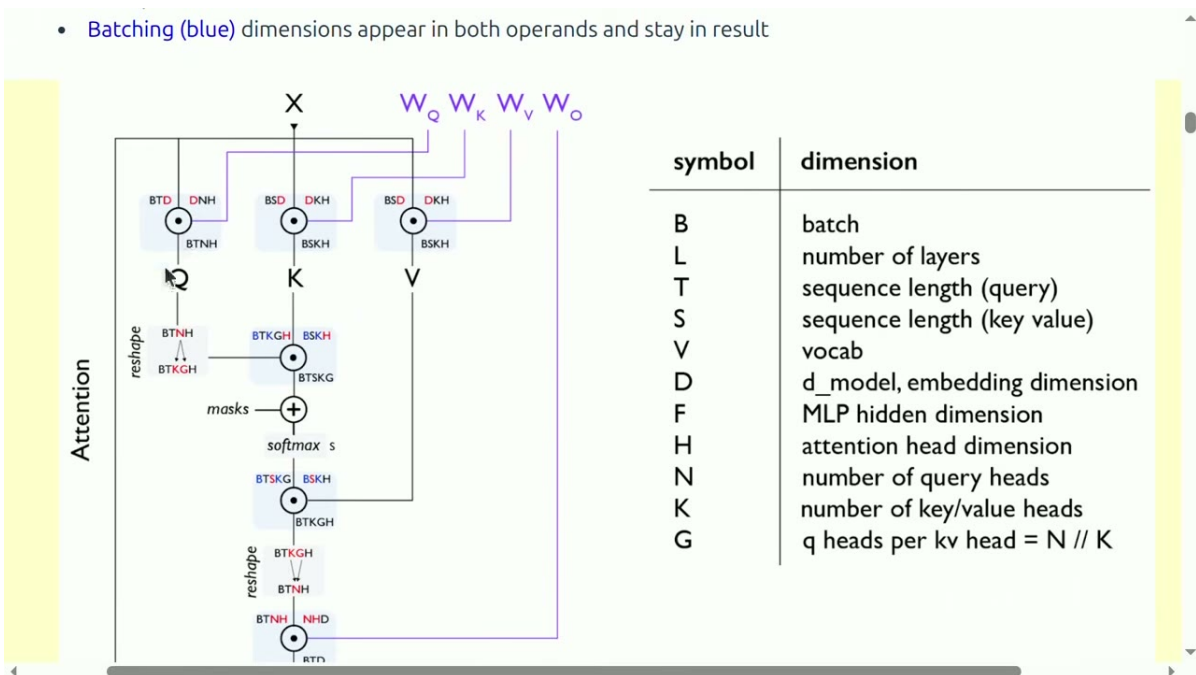


图 4: Transformer 推理张量维度记号⁴

3.2 矩阵乘 FLOPs 与高带宽内存读写核算

核心判断。“矩阵乘 FLOPs 与高带宽内存读写核算”是“Transformer 资源核算”中的一个可检验设计选择，而不是只需记忆的产品名。先用张量形状推出程序实例和掩码，再决定分块、向量化和片上复用；基准必须预热、同步并与高精度参考实现逐元素比较。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 5 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“矩阵乘 FLOPs 与高带宽内存读写核算”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁴视频画面时间区间：00:11:45–00:12:00。

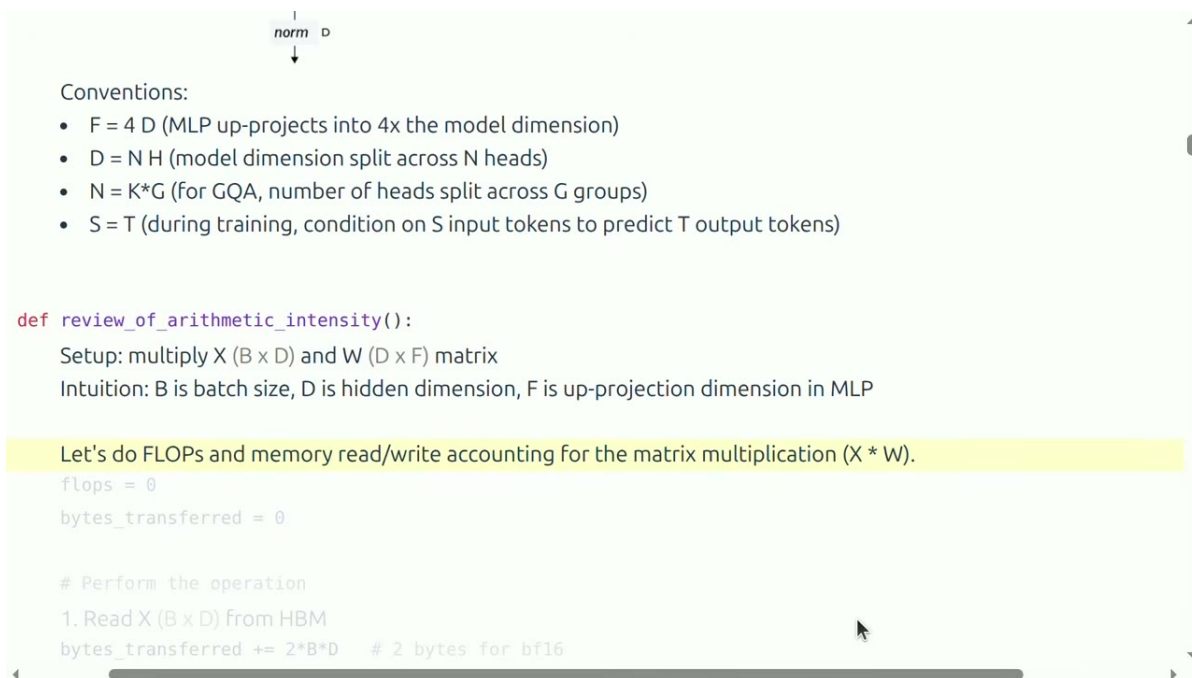


图 5: 矩阵乘 FLOPs 与高带宽内存读写核算⁵

3.3 H100 算术强度阈值约二百九十五

核心判断。“H100 算术强度阈值约二百九十五”是“Transformer 资源核算”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 6 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“H100 算术强度阈值约二百九十五”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁵视频画面时间区间：00:14:45–00:15:00。

```

Assuming B is much less than D and F, then we can simplify.
intensity = intensity.subs(D, c*B).subs(F, c*B).limit(c, oo).simplify()
assert intensity == B

Accelerator intensity of H100:
flops_per_second = 989e12
memory_bandwidth = 3.35e12
accelerator_intensity = flops_per_second / memory_bandwidth
assert round(accelerator_intensity) == 295

If computation intensity > accelerator intensity, compute-bound (good)
If computation intensity < accelerator intensity, memory-bound (bad)
Conclusion: compute-bound iff B > 295

Extreme case (B = 1, corresponding to matrix-vector product):
• Arithmetic intensity: 1
• Memory-bound (read D x F matrix, perform only 2 D F FLOPs)
• This is basically what happens with inference...

def arithmetic_intensity_of_inference():

```

图 6: H100 算术强度阈值约二百九十五⁶

3.4 本章小结

“Transformer 资源核算”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

4 KV cache 与 prefill/decode

本节路线

本节依次处理：批—矩阵向量乘受内存带宽限制、KV cache 复用历史 token 的键和值、MLP 层参数读取与激活写回。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I_{\text{prefill}} = \frac{S}{2}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁶视频画面时间区间：00:17:45–00:18:00。

$$G_3 = \frac{B_3}{C_3 + \lambda R_3 + \epsilon}$$

- B_3 是本节方法带来的质量、吞吐或覆盖收益。
- C_3 是新增计算、显存、通信、数据或标注成本。
- R_3 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_3 \leq C_{\max}, \quad R_3 \leq R_{\max}, \quad Q_3 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

4.1 批一矩阵向量乘受内存带宽限制

核心判断。“批一矩阵向量乘受内存带宽限制”是“KV cache 与 prefill/decode”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 7 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“批一矩阵向量乘受内存带宽限制”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

if computation intensity > accelerator intensity, **compute-bound** (good)
 If computation intensity < accelerator intensity, **memory-bound** (bad)
 Conclusion: compute-bound iff $B > 295$

Extreme case ($B = 1$, corresponding to matrix-vector product):

- Arithmetic intensity: 1
- Memory-bound (read $D \times F$ matrix, perform only $2 D F$ FLOPs)
- This is basically what happens with inference...

```
def arithmetic_intensity_of_inference():
```

[Scaling book chapter on inference]

图 7: 批一矩阵向量乘受内存带宽限制⁷

4.2 KV cache 复用历史 token 的键和值

核心判断。“KV cache 复用历史 token 的键和值”是“KV cache 与 prefill/decode”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 8 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“KV cache 复用历史 token 的键和值”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁷视频画面时间区间：00:20:30–00:20:45。

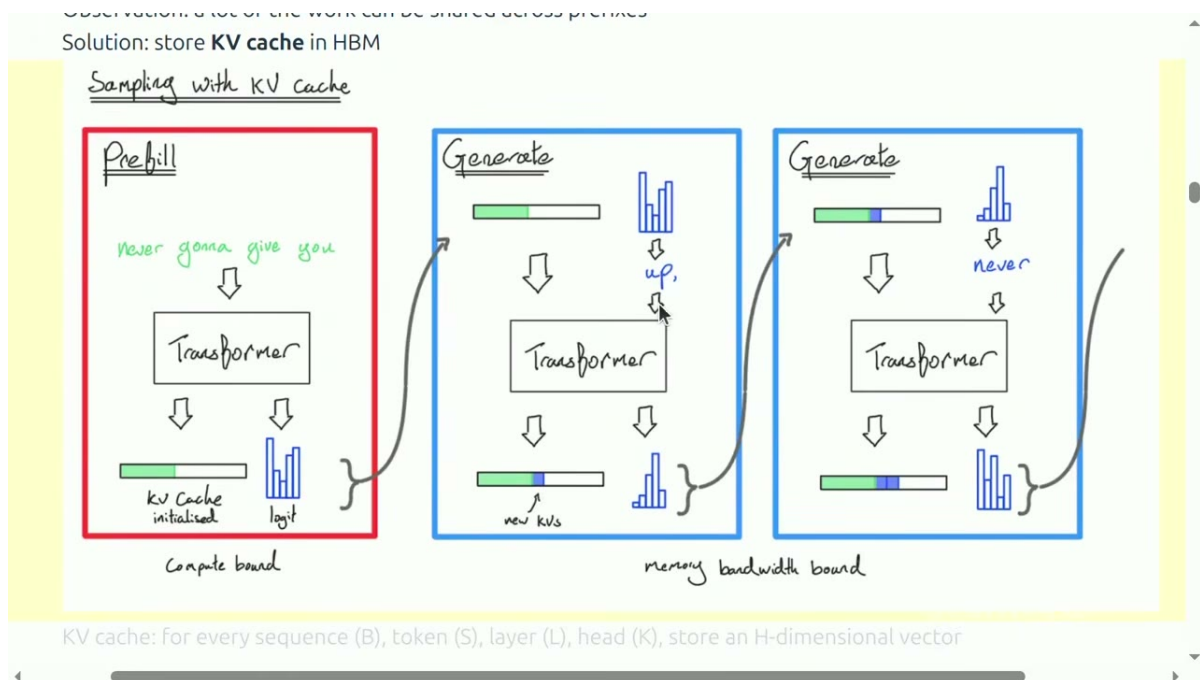


图 8: KV cache 复用历史 token 的键和值⁸

4.3 MLP 层参数读取与激活写回

核心判断。“MLP 层参数读取与激活写回”是“KV cache 与 prefill/decode”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 9 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“MLP 层参数读取与激活写回”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

⁸视频画面时间区间：00:23:30-00:23:45。

```

flops = 0
bytes_transferred = 0

# Perform the operation
1. Read X (B x T x D) from HBM
bytes_transferred += 2*B*T*D
2. Read Wup (D x F), Wgate (D x F), Wdown (F x D) from HBM
bytes_transferred += 3 * 2*D*F
3. Compute U = X (B x T x D) @ Wup (D x F)
flops += 2*B*T*D*F
4. Write U (B x T x F) to HBM
bytes_transferred += 2*B*T*F
5. Compute G = X (B x T x D) @ Wgate (D x F)
flops += 2*B*T*D*F
6. Write G (B x T x F) to HBM
bytes_transferred += 2*B*T*F
7. Compute Y = GeLU(G)*U (B x T x F) @ Wdown (F x D)
flops += 2*B*T*D*F
8. Write Y (B x T x D) to HBM

```

图 9: MLP 层参数读取与激活写回⁹

4.4 本章小结

“KV cache 与 prefill/decode”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

5 延迟吞吐和显存模型

本节路线

本节依次处理：注意力层读取 KV cache 的数据量、prefill 计算受限而 decode 内存受限、单请求延迟和批吞吐的性能模型。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$I_{\text{decode}} = \frac{S}{S+1} < 1$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

⁹视频画面时间区间：00:26:30–00:26:45。

$$G_4 = \frac{B_4}{C_4 + \lambda R_4 + \epsilon}$$

- B_4 是本节方法带来的质量、吞吐或覆盖收益。
- C_4 是新增计算、显存、通信、数据或标注成本。
- R_4 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_4 \leq C_{\max}, \quad R_4 \leq R_{\max}, \quad Q_4 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

5.1 注意力层读取 KV cache 的数据量

核心判断。“注意力层读取 KV cache 的数据量”是“延迟吞吐和显存模型”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 10 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“注意力层读取 KV cache 的数据量”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

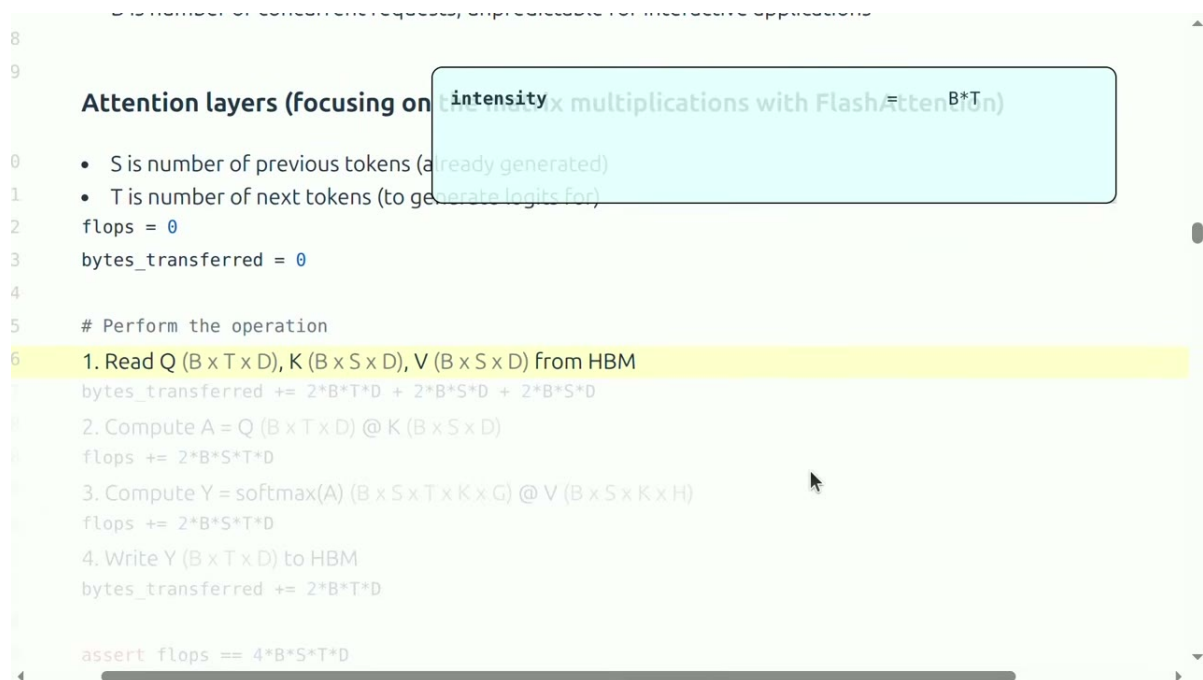


图 10: 注意力层读取 KV cache 的数据量¹⁰

5.2 prefill 计算受限而 decode 内存受限

核心判断。“prefill 计算受限而 decode 内存受限”是“延迟吞吐和显存模型”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 11 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“prefill 计算受限而 decode 内存受限”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁰视频画面时间区间：00:29:30–00:29:45。

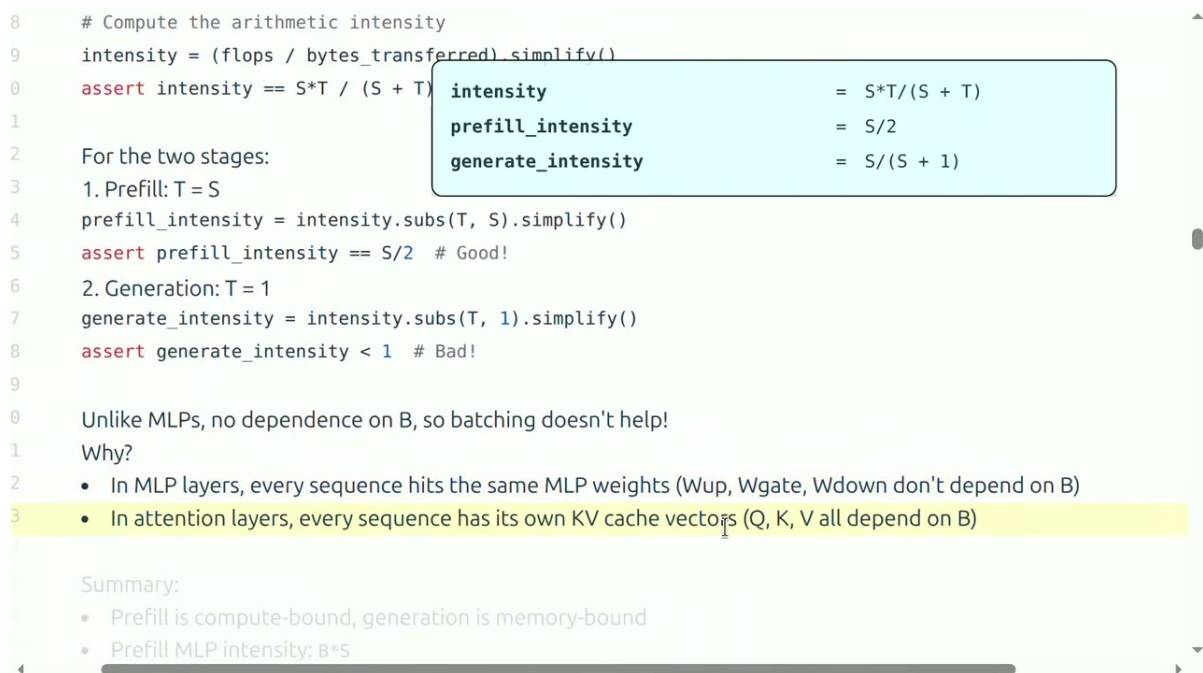


图 11: prefill 计算受限而 decode 内存受限¹¹

5.3 单请求延迟和批吞吐的性能模型

核心判断。“单请求延迟和批吞吐的性能模型”是“延迟吞吐和显存模型”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 12 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“单请求延迟和批吞吐的性能模型”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹¹视频画面时间区间：00:32:15–00:32:30。

```

323     K: 40,      # Number of key/value heads
324     H: 128,    # Head dimension
325     L: 40,     # Number of layers
326     V: 32000,  # Vocabulary size
327     memory_bandwidth: 3.35e12, # Memory bandwidth (bytes/second)
328     **args
329 }
330
331
332 def throughput_and_latency():
333     So we have shown that inference is memory-bound.
334     Let us now compute the theoretical maximum latency and throughput of a single request.
335     Assumption: can overlap compute and communication perfectly and ignore overhead.
336
337     Instantiate latency and throughput for Llama 2 13B on an H100:
338     config = llama2_13b_config()
339     stats = compute_transformer_performance_stats(config)
340
341     # Batch size 1
342     h1 = stats.substitute(B=1)

```

图 12: 单请求延迟和批吞吐的性能模型¹²

5.4 本章小结

“延迟吞吐和显存模型”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

6 GQA 与 MLA

本节路线

本节依次处理：参数和 KV cache 共同决定显存占用、Llama 十三 B 在 H100 上的延迟吞吐估算、增大 batch 改善吞吐但恶化延迟。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$M_{KV} = 2BSLKHb$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹²视频画面时间区间：00:35:15–00:35:30。

$$G_5 = \frac{B_5}{C_5 + \lambda R_5 + \epsilon}$$

- B_5 是本节方法带来的质量、吞吐或覆盖收益。
- C_5 是新增计算、显存、通信、数据或标注成本。
- R_5 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_5 \leq C_{\max}, \quad R_5 \leq R_{\max}, \quad Q_5 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

6.1 参数和 KV cache 共同决定显存占用

核心判断。“参数和 KV cache 共同决定显存占用”是“GQA 与 MLA”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 13 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“参数和 KV cache 共同决定显存占用”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

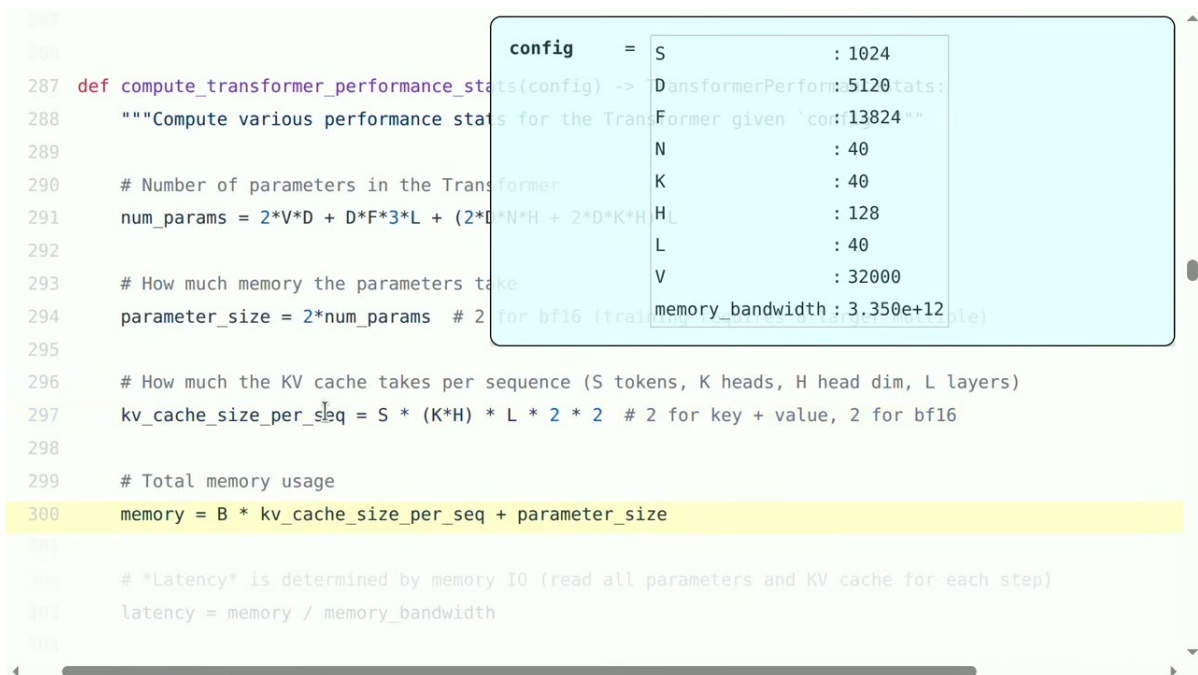


图 13: 参数和 KV cache 共同决定显存占用¹³

6.2 Llama 十三 B 在 H100 上的延迟吞吐估算

核心判断。“Llama 十三 B 在 H100 上的延迟吞吐估算”是“GQA 与 MLA”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 14 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“Llama 十三 B 在 H100 上的延迟吞吐估算”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹³视频画面时间区间：00:38:15–00:38:30。

```

298
299 # Total memory usage
300 memory = B * kv_cache_size_per_seq
301
302 # *Latency* is determined by memory
303 latency = memory / memory_bandwidth
304
305 # *Throughput* is the inverse of latency, but we're generating B tokens in parallel
306 throughput = B / latency
307
308 # Substitute config
309 num_params = num_params.subs(config)
310 memory = memory.subs(config).simplify()
311 latency = latency.subs(config).simplify()
312 throughput = throughput.subs(config).simplify()
313
314 return TransformerPerformanceStats(num_params, memory, latency, throughput)
315
316 def llama2_13b_config(args={}):

```

config

S	: 1024
D	: 5120
F	: 13824
N	: 40
K	: 40
H	: 128
L	: 40
V	: 32000
memory_bandwidth	: 3.350e+12

num_params = 13,015,449,600

memory = 838860800*B + 26030899200

latency = 0.000250406208955224*B + 0.00777041767164179

throughput = 127792.358398438*B/(32*B + 9937)

图 14: Llama 十三 B 在 H100 上的延迟吞吐估算¹⁴

6.3 增大 batch 改善吞吐但恶化延迟

核心判断。“增大 batch 改善吞吐但恶化延迟”是“GQA 与 MLA”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 15 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“增大 batch 改善吞吐但恶化延迟”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁴视频画面时间区间：00:41:15–00:41:30。

```
346 Result: worse latency, better throughput
347
348 # Batch size 256
349 b256 = stats.substitute(B, 256)
350 Result: even worse latency, even better throughput
351 h100_memory = 80e9 # H100 memory in bytes
352 assert b256.memory > h100_memory # Doesn't fit in memory!
353 Result: doesn't fit into memory and throughput gains are diminishing too...
354
355 What increasing batch size does:
356 • Worsens latency because larger KV cache (O(B) size) to read/write
357 • Improves throughput because amortizes the cost of reading parameters
358
359 Tradeoff between latency and throughput:
360 1. Smaller batch sizes yield better latency but worse throughput
361 2. Larger batch sizes yield better throughput but worse latency
362
363 Easy parallelism: if you launch M copies of the model, latency is the same, throughput increases by M!
364 Harder parallelism: shard the model and the KV cache [Scaling book chapter on inference]
```

图 15: 增大 batch 改善吞吐但恶化延迟¹⁵

6.4 本章小结

“GQA 与 MLA”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

7 量化剪枝和蒸馏

本节路线

本节依次处理：分组查询注意力减少 KV 头数、GQA 的显存收益和吞吐变化、多头潜在注意力压缩 KV 表示。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$T_{\text{latency}} \approx \frac{M_{\text{weights}} + M_{\text{KV}}}{BW}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁵视频画面时间区间：00:44:00–00:44:15。

$$G_6 = \frac{B_6}{C_6 + \lambda R_6 + \epsilon}$$

- B_6 是本节方法带来的质量、吞吐或覆盖收益。
- C_6 是新增计算、显存、通信、数据或标注成本。
- R_6 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_6 \leq C_{\max}, \quad R_6 \leq R_{\max}, \quad Q_6 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

7.1 分组查询注意力减少 KV 头数

核心判断。“分组查询注意力减少 KV 头数”是“量化剪枝和蒸馏”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 16 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“分组查询注意力减少 KV 头数”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

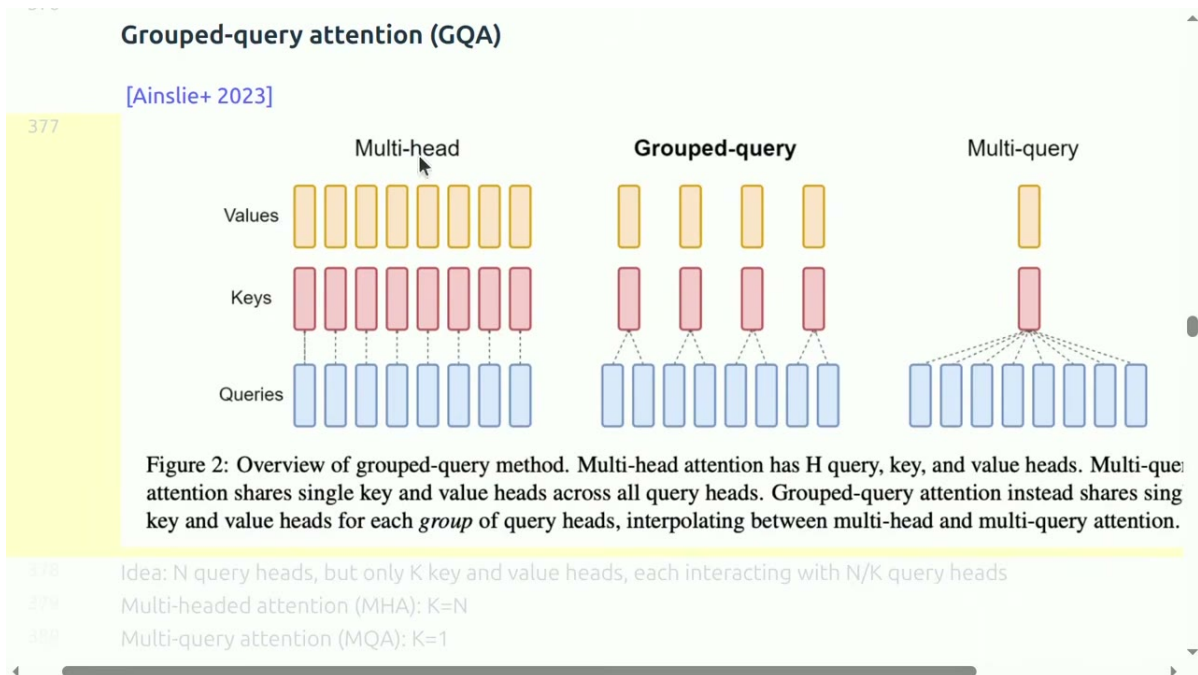


图 16: 分组查询注意力减少 KV 头数¹⁶

7.2 GQA 的显存收益和吞吐变化

核心判断。“GQA 的显存收益和吞吐变化”是“量化剪枝和蒸馏”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 17 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“GQA 的显存收益和吞吐变化”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁶视频画面时间区间：00:47:00–00:47:15。

```
385
386 Why does GQA improve latency and throughput?
387 GQA reduces the KV cache by a factor of N/K.
388 Reminder: reducing memory usage leads to speedup (since we're memory-bound)
389
390 # Original Llama 2 13B (MHA)
391 config = llama2_13b_config({K: 40, B: 64})
392 k40_b64 = compute_transformer_performance_stats(config)
393
394 # GQA with 1:5 ratio (K:N)
395 config = llama2_13b_config({K: 8, B: 64}) # Use GQA with 1:5 ratio
396 k8_b64 = compute_transformer_performance_stats(config)
397 Result: Worse latency, but better throughput (and it fits in memory now!)
398
399 # Now we can increase the batch size
400 config = llama2_13b_config({K: 8, B: 256}) # Increase batch size
401 k8_b256 = compute_transformer_performance_stats(config)
402 Result: Worse latency, but better throughput (and still fits in memory!)
```

GQA groups	
k40_b64	=
	num_params : 13,015,449,600
	memory : 79,717,990,400
	latency : 0.0238
	throughput : 2689.4807
k8_b64	=
	num_params : 11,337,728,000
	memory : 33,412,874,240
	latency : 0.0100
	throughput : 6416.6883

图 17: GQA 的显存收益和吞吐变化¹⁷

7.3 多头潜在注意力压缩 KV 表示

核心判断。“多头潜在注意力压缩 KV 表示”是“量化剪枝和蒸馏”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 18 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“多头潜在注意力压缩 KV 表示”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁷视频画面时间区间：00:50:00–00:50:15。

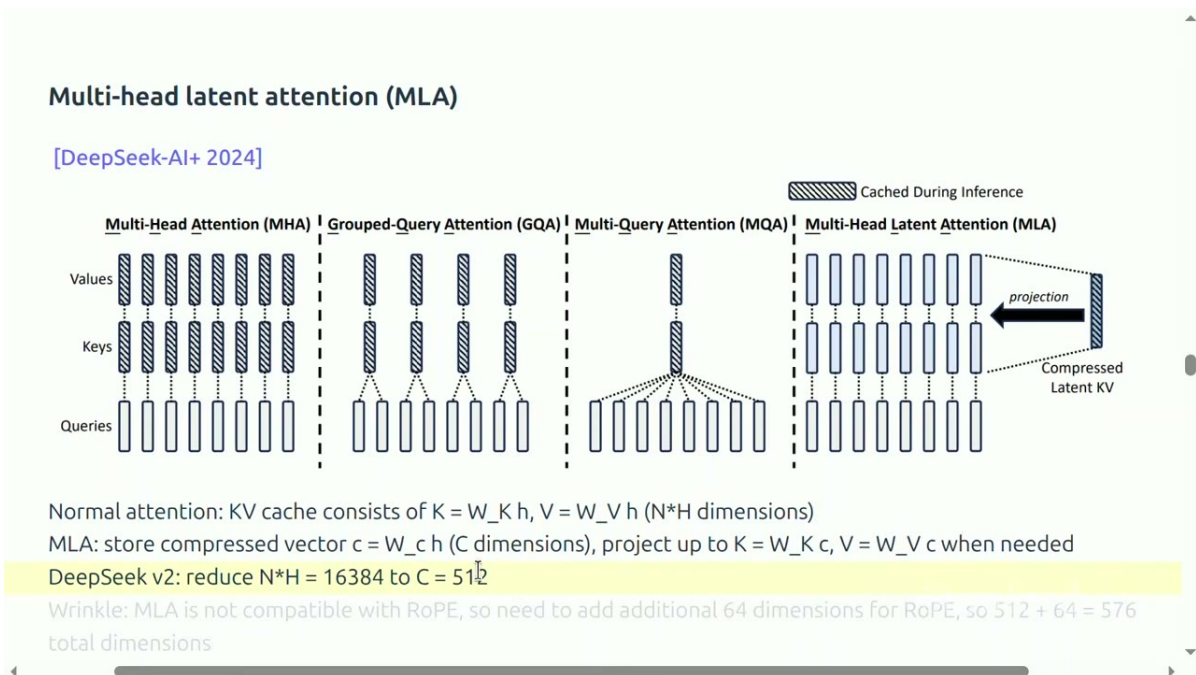


图 18: 多头潜在注意力压缩 KV 表示¹⁸

7.4 本章小结

“量化剪枝和蒸馏”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

8 推测解码

本节路线

本节依次处理：量化减少模型权重字节数、激活感知量化保留显著通道精度、推测解码由草稿模型提出多 token。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$\text{throughput} \approx \frac{B}{T_{\text{latency}}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

¹⁸视频画面时间区间：00:53:00–00:53:15。

$$G_7 = \frac{B_7}{C_7 + \lambda R_7 + \epsilon}$$

- B_7 是本节方法带来的质量、吞吐或覆盖收益。
- C_7 是新增计算、显存、通信、数据或标注成本。
- R_7 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_7 \leq C_{\max}, \quad R_7 \leq R_{\max}, \quad Q_7 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

8.1 量化减少模型权重字节数

核心判断。“量化减少模型权重字节数”是“推测解码”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 19 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“量化减少模型权重字节数”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

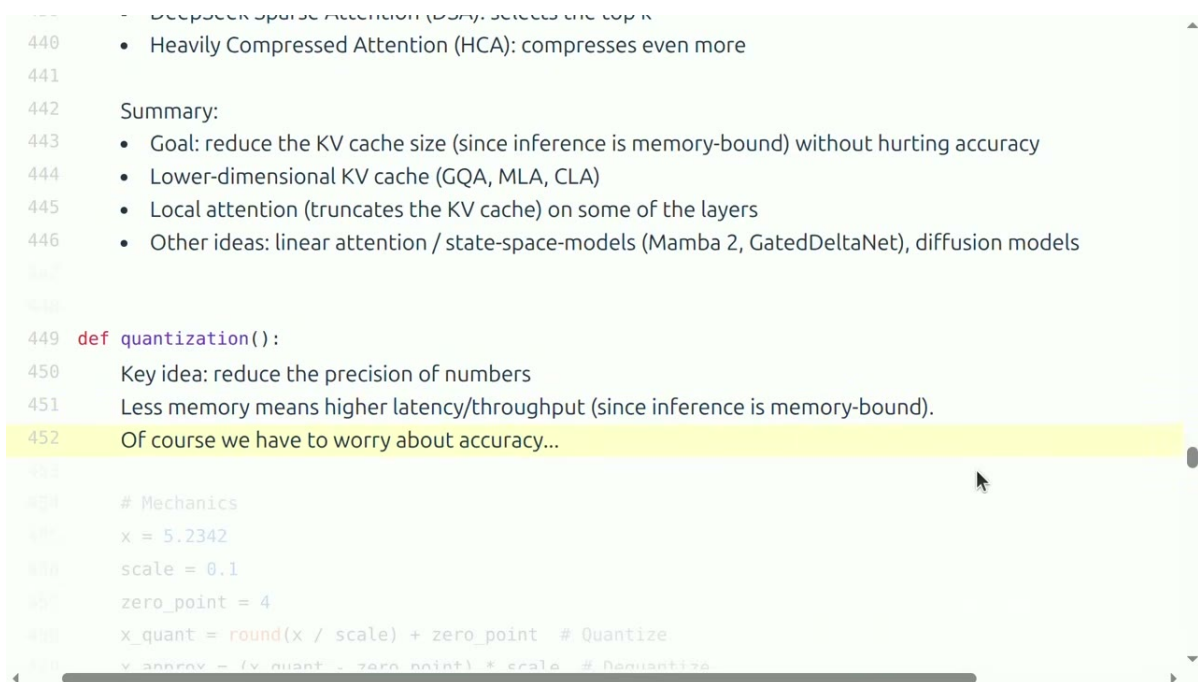


图 19: 量化减少模型权重字节数¹⁹

8.2 激活感知量化保留显著通道精度

核心判断。“激活感知量化保留显著通道精度”是“推测解码”中的一个可检验设计选择，而不是只需记忆的产品名。低精度同时改变表示范围、舍入误差和搬运字节数；需要记录尺度粒度、累加精度、异常值处理以及端到端质量，而非只记录位宽。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 20 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“激活感知量化保留显著通道精度”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

¹⁹视频画面时间区间：01:04:45–01:05:00。

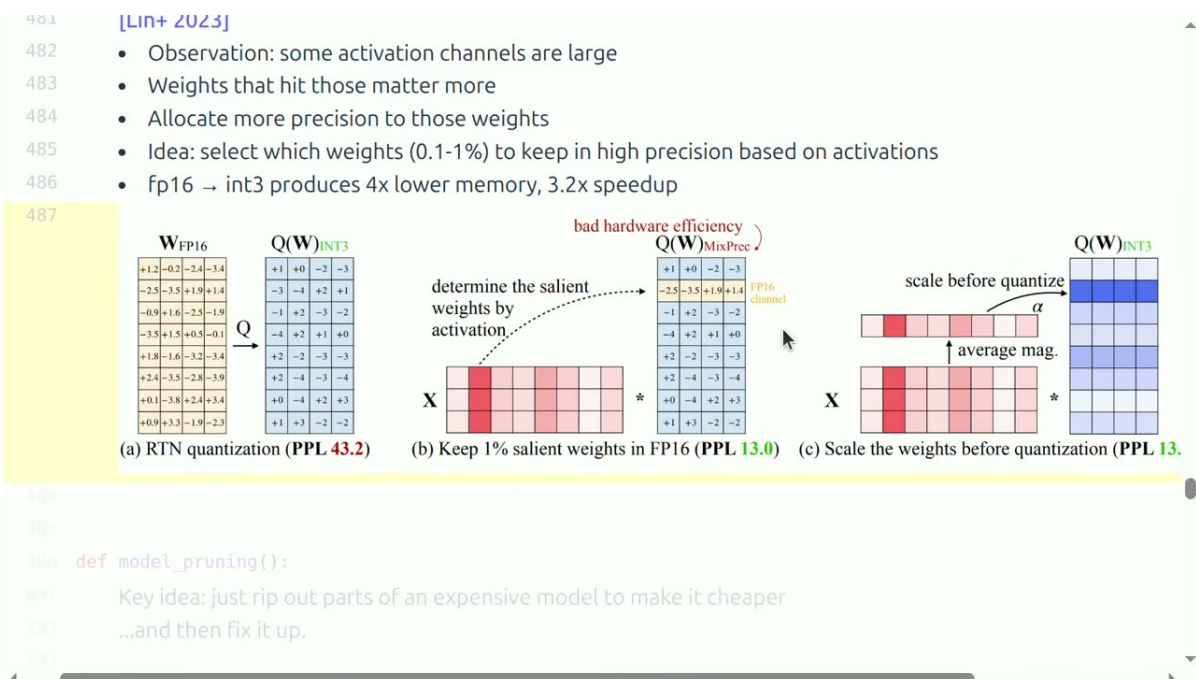


图 20: 激活感知量化保留显著通道精度²⁰

8.3 推测解码由草稿模型提出多 token

核心判断。“推测解码由草稿模型提出多 token”是“推测解码”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 21 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“推测解码由草稿模型提出多 token”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²⁰视频画面时间区间：01:07:30–01:07:45。

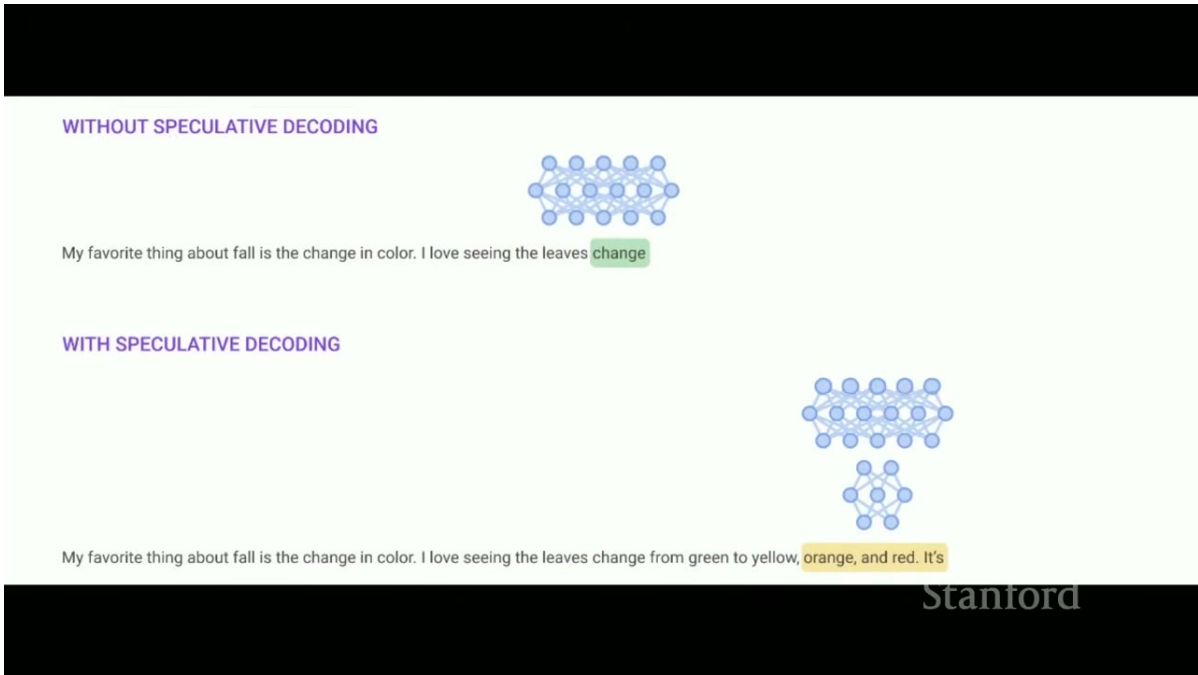


图 21: 推测解码由草稿模型提出多 token²¹

8.4 本章小结

“推测解码”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

9 连续批处理与分页注意力

本节路线

本节依次处理：草稿模型大小和接受率的权衡、连续批处理按 token 调度动态请求、分页 KV cache 支持前缀共享。它们共享一个资源与信息流边界，因而必须联合阅读，不能把其中某张图当成独立结论。

$$E[\text{speedup}] \approx \frac{E[\text{accepted tokens}]}{T_{\text{draft}} + T_{\text{verify}}}$$

- 公式中的大写字母表示张量规模、样本规模、设备数或资源总量，具体语义由本节上下文给出。
- 小写字母和希腊字母表示权重、比例、阈值、延迟或拟合参数；下标用于区分样本、设备、专家或时间步。
- 等号给出机制关系，近似号表示课程中的资源估算；实际系统还要用实测常数校准。

²¹视频画面时间区间：01:13:30–01:13:45。

$$G_8 = \frac{B_8}{C_8 + \lambda R_8 + \epsilon}$$

- B_8 是本节方法带来的质量、吞吐或覆盖收益。
- C_8 是新增计算、显存、通信、数据或标注成本。
- R_8 是错误、偏差、失稳或不可复现风险； λ 表示对风险的惩罚强度。

$$C_8 \leq C_{\max}, \quad R_8 \leq R_{\max}, \quad Q_8 \geq Q_{\min}$$

- $C_{\max}$ 和 $R_{\max}$ 分别是资源预算与风险上限。
- $Q_{\min}$ 是必须守住的质量、正确率或服务水平下界。
- 这组约束是本笔记用于复现实验决策的分析框架，并非把课程中的所有方法压缩为单一分数。

9.1 草稿模型大小和接受率的权衡

核心判断。“草稿模型大小和接受率的权衡”是“连续批处理与分页注意力”中的一个可检验设计选择，而不是只需记忆的产品名。把该主题拆成输入、变换、持久状态和输出四部分，再分别记录直接机制指标、资源成本和最终质量；若中间量不按预测变化，应优先检查实现和数据边界。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 22 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

应记住的判断

围绕“草稿模型大小和接受率的权衡”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

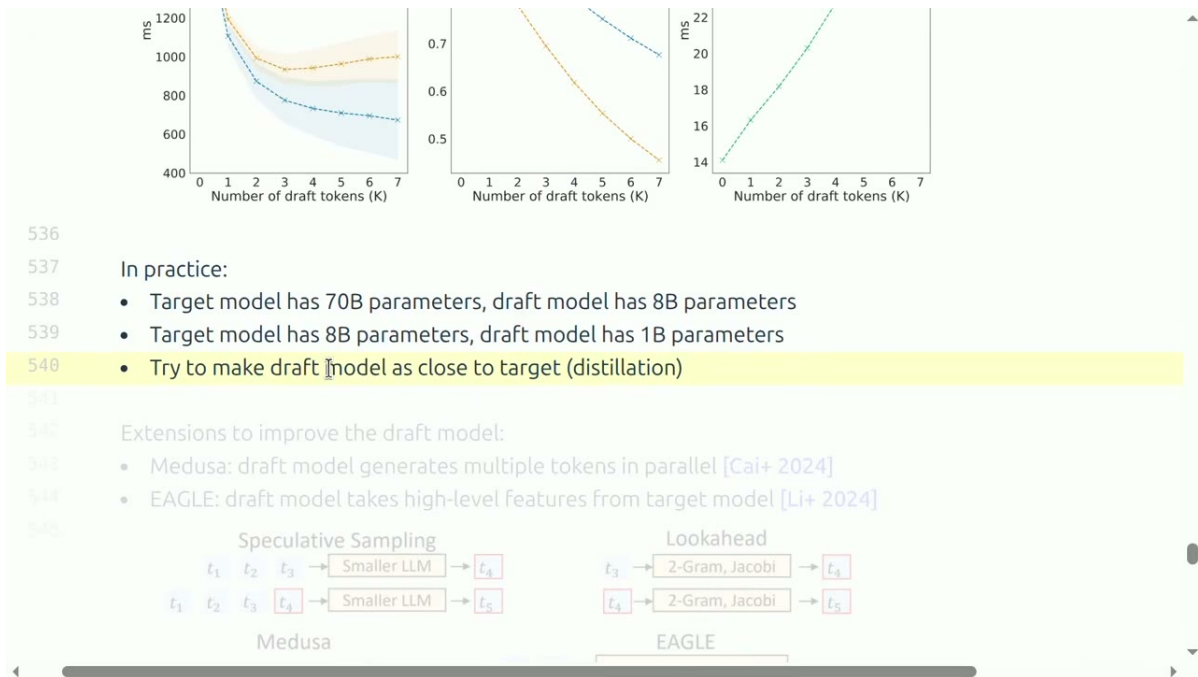


图 22: 草稿模型大小和接受率的权衡²²

9.2 连续批处理按 token 调度动态请求

核心判断。“连续批处理按 token 调度动态请求”是“连续批处理与分页注意力”中的一个可检验设计选择，而不是只需记忆的产品名。prefill 可以沿序列并行而 decode 逐 token 串行；服务评估要同时报告首 token、token 间隔、吞吐、尾延迟和显存峰值。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 23 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

复现实验记录

围绕“连续批处理按 token 调度动态请求”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²²视频画面时间区间：01:16:30–01:16:45。

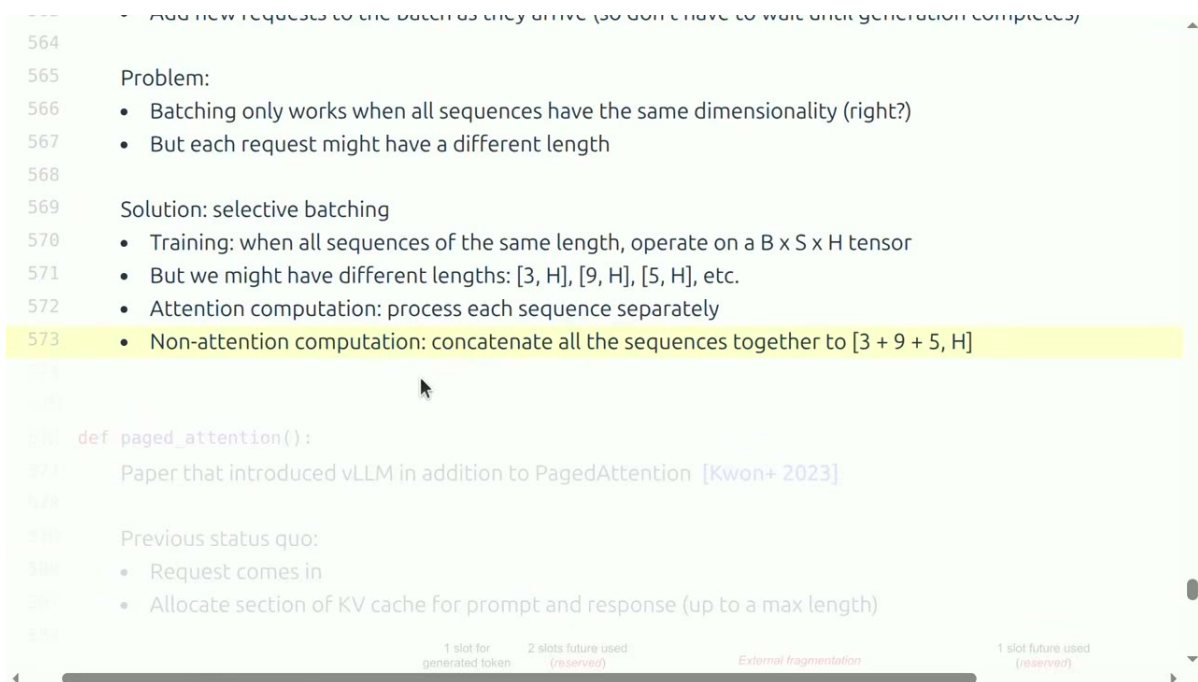


图 23: 连续批处理按 token 调度动态请求²³

9.3 分页 KV cache 支持前缀共享

核心判断。“分页 KV cache 支持前缀共享”是“连续批处理与分页注意力”中的一个可检验设计选择，而不是只需记忆的产品名。先区分 query 侧计算、历史 key/value 状态和完整注意力矩阵；核算时同时写出序列长度依赖与缓存字节数，验证时观察有效带宽、核函数时间和质量变化。

复现与诊断。实验记录至少包含输入形状或数据范围、关键状态位置、阈值与精度、资源测量和质量基线。先检查直接中间量，再检查最终指标；若最终分数改变而中间量没有按机制预测变化，应优先怀疑实现、基准或数据泄漏。第 24 个图像证据在本讲 24 个主题中的作用，是把这一判断固定到可回看的原始画面和时间区间。

常见失效模式

围绕“分页 KV cache 支持前缀共享”，至少记录直接机制指标、最终质量指标和一个反事实基线。若只给最终分数而没有中间量，就无法区分真正的机制收益、额外预算和测量误差。

²³视频画面时间区间：01:19:15–01:19:30。

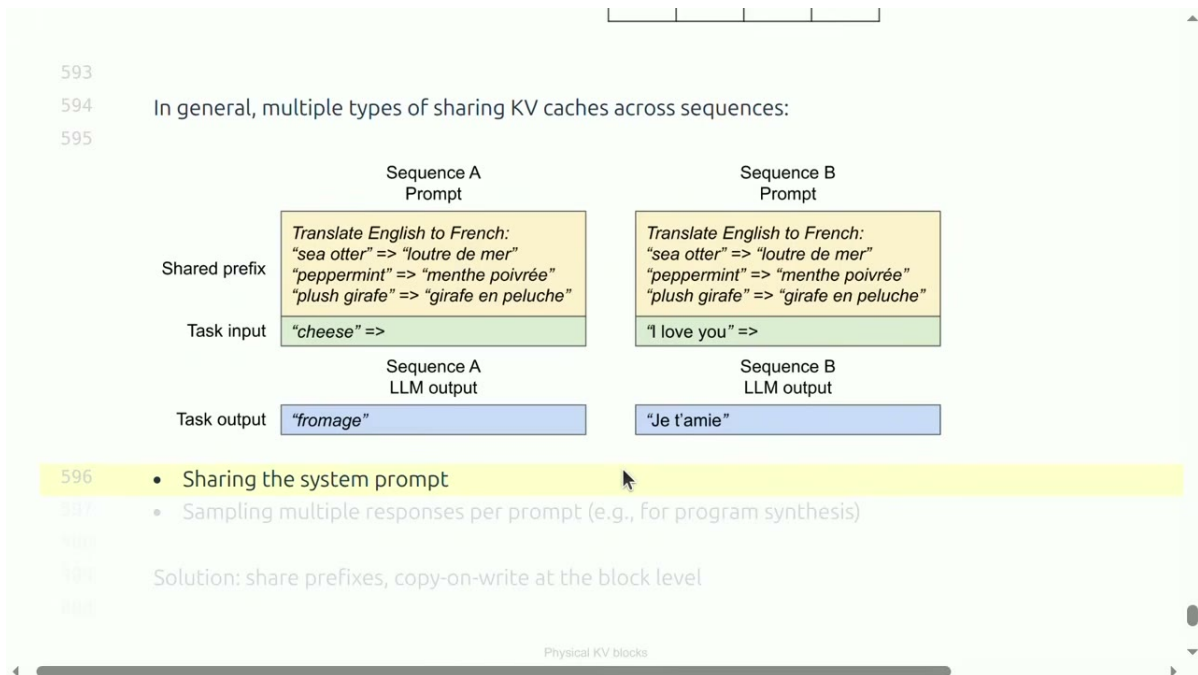


图 24: 分页 KV cache 支持前缀共享²⁴

9.4 本章小结

“连续批处理与分页注意力”的关键不是记住若干实现名称，而是建立选择顺序：先确定目标和状态边界，再核算计算、存储、通信或数据成本，随后检查本节公式的假设，最后用原始画面对应的中间量验证因果链。只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。

10 代码化复现：最小机制

下面的片段保留本讲最核心的程序结构，用于把公式和系统过程落到可执行对象。它不是生产实现；实际复现还需补充张量形状断言、数值精度、同步、随机种子、数据版本和端到端测试。

Listing 1: Lecture 10 机制伪代码

```
1 def decode_step(model, token, kv_cache):
2     logits, kv_cache = model(token, kv_cache=kv_cache, use_cache=True)
3     next_token = sample(logits[:, -1])
4     return next_token, kv_cache
```

复现检查单

先写正确性基线，再记录资源基线；每次只改变一个机制；同时保存直接中间量和最终指标；对随机算法报告多次运行；对数据与评测保存不可变版本标识。

²⁴视频画面时间区间：01:22:15–01:22:30。

10.1 本章小结

代码的价值在于暴露状态、顺序和边界。若一个概念无法写成输入、变换、状态、输出和断言，它通常还没有被真正理解。

11 总结与延伸

一页压缩

本讲的主问题是：为什么自回归 decode 通常受内存带宽限制，架构和服务系统又怎样共同降低每个输出 token 的成本？统一答案是：推理优化同时处理模型内的算子、KV cache 的状态和在线请求的调度；单点微优化不能替代端到端核算。

第一，优先理解机制而不是产品名。模型、硬件、数据和评测不断更新，但张量形状、状态位置、目标函数、通信边界和测量误差仍然是稳定的分析对象。第二，所有优化都要写出代价守恒：减少某一种成本通常会增加另一种成本或风险。第三，把小规模实验推广到大规模之前，必须检查瓶颈是否改变、数据分布是否改变、实现常数是否仍在同一数量级。第四，最可靠的课程笔记应允许读者沿时间脚注回到原视频，并沿官方材料找到公式或代码出处。

开放问题

只优化平均吞吐会伤害尾延迟；只压缩权重会遗漏 KV cache；草稿模型很快也不代表推测解码一定有收益。后续实验应把这一风险写成可观测量和停止条件，而不是留在讨论部分。

11.1 本章小结

完成本讲后，读者应能用统一语言解释每个主题的输入、状态、变换、输出、资源成本和失败模式，并据此设计一个含基线、消融、误差条和证据记录的最小实验。

12 资料来源与审计边界

- 课程主页：<https://cs336.stanford.edu/>。
- 视频：<https://www.youtube.com/watch?v=EfM546A79aM>。
- Stanford 官方材料：lecture_10.py；客座讲座若无官方文件则以视频和字幕为主。
- 本地证据：audio.srt、figure_manifest.tsv、figure_verification.txt、teaching_atoms.tsv、numerical_claims.tsv。

12.1 本章小结

本笔记覆盖播放列表中的本讲内容与课程主页当日可取得的官方材料。没有官方讲义的客座讲座不会被伪装成有讲义；讲者口头推测、本笔记分析框架和官方公式在文字中分别标明。